
CSE8603

Web Application Development

Student pack: Attestations, SIS and TSIS (lecture side)

Autumn semester 2026-2027 · Year 2 · Language of instruction: English

Lecturer: Temirlan Shoiynbek (Shoiynbek T.A.), Senior Lecturer

Contact: shoinbektima@gmail.com

Practice instructor: Zhanegizov A.S., Senior Lecturer (practice scores are separate)

Target stack: React · Node/Express · PostgreSQL (SQLite allowed if Postgres is hard)

Prerequisite flavor: CSE6624 / CSE6621 web fundamentals

Important scope note

Temirlan teaches LECTURES. Weekly Practice (28 points) is graded by the practice instructor and is NOT a weekly workbook in this PDF. This pack covers SIS (6), TSIS (6), Attestation 1 (9) and Attestation 2 (9) on the lecture assessment side. Final exam (40) is a separate project.

Chill classroom rule

No live-coding humiliation. AI is allowed if you can explain and slightly modify your work. Anti-copy rules (seed, niche, screenshots, defence, unique README) are mandatory.

1. How to use this booklet

This is the lecture-side assessment guide. Keep one monorepo or two linked repos (client + server). Claim your niche early. TSIS weeks: 2, 5, 8, 11. SIS weeks: 3, 6, 10, 13. Attestation 1 ~ week 8 (React prototype). Attestation 2 ~ week 15 (API + DB sketch / mini CRUD).

- Feasible for lecture-course students: Att1 can be a React SPA with mock data; Att2 can be Express + Postgres/SQLite with clear steps (full production deploy not required).
- Practice weekly scores come from the practice instructor; do not expect this PDF to list them.
- Submit GitHub link + zip (+ LMS if announced).

2. How scoring works (this pack)

Component	Points	Notes
Activeness in lectures	2 total	In-class; brief note only here
TSIS (with lecturer)	$4 \times 1.5 = 6$	Weeks 2, 5, 8, 11
SIS (independent)	$4 \times 1.5 = 6$	Weeks 3, 6, 10, 13
Practical tasks	28	From practice instructor; NOT detailed here
Attestation 1	9	~ Week 8 · React niche prototype
Attestation 2	9	~ Week 15 · API + DB mini CRUD
Final exam*	40	Separate project brief

Late policy: –10% of the item if late without documented excuse; beyond one week may be zero.

AI policy: helpers OK; defence requires explain + modify. Blind paste of a full tutorial app with only renamed titles is not enough.

3. Anti-copy rules (critical)

- **(a)** Personal seed (Hue/Accent) from ID2 / month / surname letter.
- **(b)** Unique niche from the CSE8603 list (different from CSE6624 list).
- **(c)** Screenshots: VS Code (or Cursor) with your component/route open + browser DevTools; watermark or name visible in UI/comment.
- **(d)** Oral defence: 3 random questions, ~2 minutes.
- **(e)** GitHub README: full name + ID + niche + seed + how to run client/server.
- **(f)** Deductions for identical component trees, CSS modules, or Express route files to a peer.

Warning

Forking a classmate repo and changing niche strings still looks the same in structure. Temirlan will compare component names, folder layout, and API shapes across the group.

4. Seed, niches, submit

Your personal seed (anti-copy)

Same spirit as Year 1, different niche list. Compute once; put in README and UI.

ID2 = last 2 digits of student ID. **M** = birth month (1-12). **L** = first letter of surname (A=1 ... Z=26, Latin transliteration).

1. Hue = $(ID2 \times 11 + M \times 17) \bmod 360$; **Accent** = $(Hue + 55 + L) \bmod 360$.

2. Niche index = $(ID2 \times 3 + M + L) \bmod 24 \rightarrow$ niche number = index + 1 (if taken, next free; document in README).

3. Watermark: CSE8603 | FULL_NAME | STUDENT_ID | niche-N

4. Theme tokens in React (CSS variables or a theme.js) must use Hue/Accent.

RU tip

Формула другая, чем в CSE6624, и список niche другой. Не копируйте чужой Attestation 1 из прошлого года: стек React + API проверяется на защите.

Niche list for CSE8603 (one per student)

Niches A	Niches B
1. Campus lost-and-found tracker	13. Study-with-me session finder
2. Almaty coworking room booking	14. Alumni mentorship requests
3. Student freelancers marketplace	15. Micro-climate plant care logs
4. Hackathon team matcher	16. Second-hand textbook exchange
5. Lab equipment reservation	17. Short film festival submissions
6. Internship diary / logbook	18. Accessibility issues reporter (campus)
7. Open-source contribution board	19. Sports pickup-game organizer
8. Local NGO volunteer shifts	20. Research paper reading club
9. Peer tutoring scheduler	21. Startup idea feedback wall
10. Recipe share for dorm cooking	22. Multilingual phrase flashcards
11. Bike parking & repair network	23. Eco-challenge habit tracker
12. Campus event ticketing (mini)	24. Capstone project partner finder

How to submit

1. GitHub repo (private with collaborator invite, or public).
2. README: run instructions for client and server; include watermark string.
3. Zip without node_modules and without huge DB dumps.
4. Submit link + zip **as announced by the instructor** (LMS / email / form).
5. Questions: shoinbektima@gmail.com with subject CSE8603 + your group.

Semester map (this pack only)

Use this as a planning board. Exact class dates follow the university calendar; week numbers match the syllabus assessment table.

Week	This pack	Suggested focus
1	Activeness note	Vite/React hello + niche claim
2	TSIS-1 (1.5)	Tooling & repo check
3	SIS-1 (1.5)	React niche one-pager
4	Practice (other instructor)	Follow practice brief
5	TSIS-2 (1.5)	Components & state
6	SIS-2 (1.5)	Stateful mock list
7	Routing draft	Att1 pages
8	TSIS-3 + Attestation 1 (9)	React prototype due
9	Practice (other instructor)	API intro in lectures
10	SIS-3 (1.5)	Router + form validation
11	TSIS-4 (1.5)	Express/DB sketch
12-13	SIS-4 (week 13)	API.md + fetch states
14	Wire client to API	CRUD polish
15	Attestation 2 (9)	API + DB mini CRUD due
Exam period	Exam project (40)	Separate brief

RU tip / Планирование

Отметьте дедлайны в телефоне. TSIS нельзя 'сдать файлом вместо себя' без договорённости: нужен короткий контакт с преподавателем.

Worked example: computing a seed

Fake student: **Dias Nurpeisov**, ID ending **08**, born in **November (M=11)**, surname letter **N → 14**.

- Hue = $(8 \times 11 + 11 \times 17) \bmod 360 = (88 + 187) \bmod 360 = 275$
- Accent = $(275 + 55 + 14) \bmod 360 = 344$
- Niche index = $(8 \times 3 + 11 + 14) \bmod 24 = 49 \bmod 24 = 1 \rightarrow$ niche **2. Almaty coworking room booking**
- Watermark: CSE8603 | Dias Nurpeisov | 20XXXXXX08 | niche-2

```
// theme.js
export const theme = {
  primary: 'hsl(275, 65%, 40%)',
  accent: 'hsl(344, 70%, 45%)',
  watermark: 'CSE8603 | Dias Nurpeisov | 20XXXXXX08 | niche-2',
};
```

Do this on page 1 of your README

Paste your calculated Hue, Accent, niche number, and watermark. If numbers look wrong, show your arithmetic.

README template and screenshot protocol

Copy this skeleton into README.md and fill the brackets. English is fine; Russian notes optional.

```
# CSE8603 - [Full Name] - [Student ID]
Niche: [#] [name]
Seed: Hue=[..], Accent=[..]
Watermark: CSE8603 | [Full Name] | [ID] | niche-[#]

## Run client
cd client && npm install && npm run dev

## Run server (Att2)
cd server && cp .env.example .env && npm install && npm run dev

## API
See API.md and AUTH_NOTES.md
```

Screenshot protocol (required evidence)

1. In the editor: open your main file; zoom so your name in a comment OR the watermark string in code is readable.
2. Capture the whole window (OS screenshot).
3. In the browser: open DevTools (F12). Elements tab with the watermark node selected, OR Console with a `console.log` of your watermark, OR Network tab for Att2 API calls.
4. Save as `screenshots/sis1-vscode.png` and `screenshots/sis1-devtools.png` (rename per task).
5. Blur unrelated private browser tabs if needed; do not blur your name/ID/watermark.

RU tip / Скриншоты

Если watermark мелко, увеличьте страницу (Ctrl/+). Преподаватель должен прочитать строку без лупы.

Suggested folder structure

```
cse8603-[surname]/
  README.md  API.md  AUTH_NOTES.md
  screenshots/
  client/    # Vite React
    src/components  src/pages  src/data  src/theme.js
  server/    # Express (Att2)
    src/routes  src/db  .env.example
  sis/       # optional per-week snapshots
```

Part A - SIS (4 × 1.5 = 6)

Weeks 3, 6, 10, 13 · independent · ~1-2 hours each · niche + seed required

SIS-1 · Week 3 · Niche product one-pager in React (1.5)

Create a Vite + React app with a single marketing/landing view for your niche: hero, 3 feature cards, CTA button. Use your seed colors via CSS variables. No backend yet.

- Components: at least Header, Hero, FeatureGrid, Footer.
- Watermark in footer; README with name/ID/niche/seed.
- Screenshot of component file + running browser DevTools.

Criterion	Pts
Working React app + components	0.6
Seed theme + niche content	0.5
README + screenshots	0.4

RU tip

`npx create-vite@latest` → React → CSS variables в `:root` → соберите 4 компонента. Не тащите Redux на этом шаге.

SIS-2 · Week 6 · Stateful list UI (mock data) (1.5)

Add a page/section that renders a list from a mock array (your niche entities). Support add + delete (or archive) with React state. Controlled input for the form.

- Keys on list items; no console errors.
- Empty state message when list is empty.
- Optional: filter text box.

Criterion	Pts
State add/delete works	0.7
UX clarity (empty state, niche labels)	0.4
Anti-copy evidence	0.4

SIS-3 · Week 10 · Client routing + form validation (1.5)

Use React Router (or equivalent) for at least Home / List / Detail-or-New routes. Build a form with client-side validation messages (required fields, simple email/url check).

- Navigation highlights current route.
- Validation errors shown near fields; submit blocked until valid.
- Detail view can use mock id from the list.

Criterion	Pts
Routing works (3 views)	0.6
Form validation UX	0.5
README/screens/watermark	0.4

SIS-4 · Week 13 · API contract sketch + fetch to mock/server (1.5)

Write a short API.md describing 4 endpoints for your niche (method, path, example JSON). Then either (a) fetch from a local Express stub you start writing, or (b) fetch from a public JSON placeholder shaped to your niche. Render loading / error / success states.

Criterion	Pts
Clear API.md with 4 endpoints	0.5
Fetch + loading/error/success UI	0.6
Evidence + niche personalization	0.4

Part B - TSIS (4 × 1.5 = 6)

Weeks 2, 5, 8, 11 · short checks with lecturer · show progress, 2 questions, 1 demo

Same chill format as Year 1: progress over perfection. Bring running app or screen share.

TSIS-1 · Week 2 · Tooling & repo (1.5)

Demo: Show Vite/React hello app, GitHub README draft with niche/seed, folder structure.

Sample questions (pick 2):

- What does the bundler do for you?
- Where will client and server live?
- State your niche in one sentence.

Rubric: progress 0.5 · answers 0.5 · demo 0.5

TSIS-2 · Week 5 · Components & state (1.5)

Demo: Demo props + useState on a niche widget (counter, tag picker, or tiny form).

Sample questions (pick 2):

- Props vs state?
- Why keys in lists?
- How are seed colors applied?

Rubric: progress 0.5 · answers 0.5 · demo 0.5

TSIS-3 · Week 8 · Att1 walkthrough (1.5)

Demo: Show Attestation 1 routes and mock data; point to personalization.

Sample questions (pick 2):

- Which components are reusable?
- How would this talk to a real API?
- What is still mock?

Rubric: progress 0.5 · answers 0.5 · demo 0.5

TSIS-4 · Week 11 · Server sketch (1.5)

Demo: Show Express (or plan) with one GET and one POST route; or DB schema draft on paper/README.

Sample questions (pick 2):

- What is middleware?
- SQL injection: how will you avoid it?
- Postgres vs SQLite: your choice and why?

Rubric: progress 0.5 · answers 0.5 · demo 0.5

Band	Meaning	Pts
Full	On time, clear demo, seed/niche visible, solid answers	1.5
Partial	Weak demo or vague answers	0.5-1.0
Minimal	Absent / cannot run anything / no explanation	0-0.5

Part C - Attestation 1 (9 points)

React front-end prototype for your unique niche · mock data OK · ~ week 8

Deliver a React SPA that feels like a real product shell for your niche. Backend can be mocked (JSON file, in-memory module, or hard-coded module). Focus on components, routing, state, and clear UX.

Requirements checklist

- ☐ Vite + React (function components).
- ☐ At least 4 routes/views (e.g. Home, Catalog/List, Item Detail, Create/Edit form).
- ☐ Shared layout: nav + footer with watermark.
- ☐ Seed-based theme tokens visible in UI.
- ☐ List from mock data with search or filter.
- ☐ Form with validation; updates mock state (or localStorage) so the list reflects changes in-session.
- ☐ Responsive enough to use on a phone-width window.
- ☐ README: name, ID, niche, seed, scripts to run; screenshots folder.
- ☐ Clean structure: components/, pages/, data/ (or similar).

RU tip / Реалистичный минимум

Не нужно Firebase и не нужно Redux. useState + React Router + mock массив. Главное: уникальный niche-контент и чтобы вы объяснили код на защите.

Rubric out of 9

Criterion	Pts	Notes
App structure, routes, reusable components	2.5	Not one giant App.js blob
List + filter/search + form validation	2.5	Works in demo
Niche UX + seed theme + responsive basics	1.5	Visibly yours
README, screenshots, watermark, repo quality	1.5	Anti-copy
Defence (3 questions)	1.0	Explain + modify

Grade bands

Band	Range	Meaning
Excellent	8.0-9.0	Polished prototype, confident defence
Good	6.5-7.9	Complete with small gaps
Passable	4.5-6.4	Runs but shallow routing/state/personalization
Weak	0-4.4	Broken, cloned, or failed defence

Defence bank (pick 3)

1. Trace a click from nav to the page component.
2. Where does mock data live? How would you replace it with fetch?
3. Show form validation; break it on purpose then fix.
4. Which state is lifted and why?
5. Change accent color token live.
6. How do you avoid prop-drilling pain later?
7. What accessibility concern exists in your form?
8. Compare your folder structure to a peer's (without copying).
9. What will Attestation 2 add behind this UI?
10. Point to AI-assisted code you still fully understand.

Part D - Attestation 2 (9 points)

API + DB sketch or working mini CRUD · Express + PostgreSQL or SQLite · ~ week 15

Connect the front end to a small back end. Goal: a believable mini CRUD for your niche entities plus short auth notes (full production auth optional). Keep it feasible.

Requirements checklist

- ☐ Express (or equivalent) HTTP API with at least: list, get-one, create, update-or-delete.
- ☐ PostgreSQL preferred; **SQLite is accepted** if Postgres install is painful (document why).
- ☐ One table (or two) with a clear schema in README; use parameterized queries / ORM safely.
- ☐ React client calls the API (CORS configured); loading and error states shown.
- ☐ Auth notes: either a minimal demo (e.g. HTTP-only cookie session / JWT prototype / fixed API key for local demo) OR a written AUTH_NOTES.md explaining how you would add register/login securely (must be specific, not vague slogans).
- ☐ Seed/niche personalization still present; new screenshots of server terminal + client + DevTools Network tab.
- ☐ .env.example committed; secrets not committed.

Feasible path

If time is short: SQLite + better-sqlite3 or Prisma SQLite; 1 resource CRUD; AUTH_NOTES.md with password hashing + session outline. Working > over-engineered half-broken.

RU tip

Сначала таблица и POST/GET в Thunder Client/Postman, потом подключите React. Не начинайте с Docker Kubernetes на этом аттестате.

Rubric out of 9

Criterion	Pts	Notes
API CRUD completeness & correct status codes	2.5	Demo with Network tab
DB schema + safe queries	2.0	Postgres or SQLite
Client integration (fetch states)	2.0	Not hardcoded after claim of API
Auth notes or minimal auth demo + secrets hygiene	1.0	AUTH_NOTES.md OK
Personalization + screenshots + README runbook	0.5	Anti-copy
Defence (3 questions)	1.0	Explain + modify

Defence bank (pick 3)

1. Show CREATE then LIST in Network tab.
2. Where is SQL/parameter binding? Why not string concat?
3. What happens when the server is down?
4. Explain your schema fields for the niche.
5. Walk through AUTH_NOTES: password storage?
6. CORS: what did you allow and why?
7. Migrate one field: how would you change schema?
8. Difference between 401 and 403 in your design?
9. What would you deploy first on a free host?
10. Which bug did you hit while wiring client to API?

Starter snippets (ideas, not copy-paste finals)

Illustrative only. Rename everything to your niche.

Express GET list sketch

```
app.get('/api/rooms', async (req, res) => {
  try {
    const rows = await db.all('SELECT * FROM rooms ORDER BY id DESC');
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: 'Failed to load rooms' });
  }
});
```

React fetch with states

```
const [items, setItems] = useState([]);
const [status, setStatus] = useState('idle');
useEffect(() => {
  setStatus('loading');
  fetch('http://localhost:3001/api/rooms')
    .then(r => { if (!r.ok) throw new Error('bad'); return r.json(); })
    .then(data => { setItems(data); setStatus('ok'); })
    .catch(() => setStatus('error'));
}, []);
```

AUTH_NOTES.md skeleton

```
# Auth notes (CSE8603)
Goal: email+password register/login.
Store password hash with bcrypt (cost 10+).
Session: HTTP-only cookie OR JWT in memory (not localStorage if possible).
Protect write routes with middleware.
Never commit .env secrets.
```

Make your work look different from peers

Even with different niches, students sometimes share the same tutorial skeleton. Avoid the patterns below.

- Do not reuse the same class names like `card1` `card2` `card3` from a shared zip.
- Rename components/sections to niche words (e.g. `FilmNightHero`, `CoworkBookingForm`).
- Change layout rhythm: card count, nav labels, footer content, image subjects.
- Write your own copy (3-5 sentences per section). Translating the same English paragraph still looks cloned if structure matches.
- JS: different function names and control flow; do not paste the same quiz engine with only question text swapped.

FAQ

Q: Can I use Bootstrap in Attestation 1 (CSE6624)?

A: Yes, optional. Attestation 2 expects Bootstrap components used seriously.

Q: Can I use TypeScript / Next.js (CSE8603)?

A: Plain React + Vite is the default. Extra tools are OK only if you can defend them and keep the project runnable.

Q: Private GitHub repo?

A: Yes. Invite the instructor or send a zip that matches the repo state.

Q: Missed a TSIS week?

A: Email ASAP with reason. Makeup is not automatic; instructor decides.

Q: AI wrote half my code.

A: Allowed if you understand it. Expect a modify-request in defence.

Q: Postgres will not install.

A: CSE8603 Att2: use SQLite and say so in README. Still need a real DB file and queries.

Q: Practice points?

A: CSE6624: weekly journal. CSE8603: practice instructor. Not regraded in this booklet.

Q: Exam project?

A: Separate 40-point brief. Do not treat Att2 as the exam.

2-minute defence prep sheet

Practise aloud once. Timer on phone. Structure:

1. 20 sec: My niche is ...; my seed hue is ...; watermark is in the footer.
2. 40 sec: Demo the main happy path (form submit / add item / API create).
3. 40 sec: Open one code file and explain the key function.
4. 20 sec: Answer the instructor's modify request (color, label, tiny bug).

Chill reminder

Nervous is OK. Reading from the screen without understanding is not OK. If stuck, say what you tried; that still scores better than silence.

Mini drill ideas (pick one before Att2)

- Add a field to the JSON body and show it in the list.
- Return 400 when a required field is missing.
- Toggle a filter chip component without breaking keys.

Integrity & identical-work deductions

Temirlan may compare repos across the group. Typical deductions (applied on top of rubrics):

Issue	Typical impact
Same HTML/CSS skeleton as peer (diff text only)	−30% to −100% of item
Same JS functions/class names as peer tutorial zip	−20% to −50%
Missing watermark / screenshots / README identity	−0.5 to −1.5 on item
Cannot explain or modify in defence	Defence criterion → 0; item may fail

Issue	Typical impact
Copied from prior year without rewrite	Item may be zeroed

If you collaborated ethically (pair discussion), write a short note in README: who did what. Each student still needs their own niche and seed.

5. Reminders

Submit checklist

- ☐ README: full name, student ID, niche, seed, how to run client & server.
- ☐ Watermark visible in UI.
- ☐ Screenshots include VS Code/Cursor + browser DevTools (Network for Att2).
- ☐ No node_modules / no secret keys in git.
- ☐ I can explain and modify my code in ~2 minutes.
- ☐ Structure is not a clone of a classmate project.

Out of scope for this PDF

- Weekly Practice (28 pts): scored by practice instructor Zhanegizov A.S.; follow their brief.
- Lecture activeness (2 pts): participate in lectures.
- Final exam project (40 pts): separate brief.

Good luck

Ship a small niche app you can demo proudly. Contact: shoinbektima@gmail.com (CSE8603 + group).

© Satbayev University · CSE8603 · Lecturer Temirlan Shoiynbek · Autumn 2026-2027 · For enrolled students. Not a substitute for the official syllabus.